Learning Summary Report

Name: Yihao Yu

Student ID: 35567082

Date: 20 October 2026

Self-Assessment

Grade demonstrated: 4K

Declaration

I declare that this portfolio is my individual work. I have not copied from any other student's work or from any other source except where due acknowledgment is made explicitly in the text, nor has any part of this submission been written for me by another person. The extent to which I have used generative AI is described and appropriately attributed.

Signature: Yihao Yu

Statement of Achievements

I am applying for a 4K grade in FIT2085. My portfolio demonstrates progress from basic C++ programming and sorting to modular solutions using linked lists, stacks, queues, trees, heaps and hash tables. The P, C, D and HD tasks provide evidence of implementation, complexity analysis and correctness reasoning.

My completed 4K tasks are Faulty Check-in and Perfect Test. The reports in my 4K portfolio each contain an AC verdict and a score of 100/100, together with handwritten analysis and a new problem statement. Their folders also contain C++ source and video recordings. Faulty Check-in uses a queue and a stack in O(n) total time; Perfect Test uses a min heap in O(n log n) time. These tasks support my ability to select suitable strategies, justify their behaviour and transfer the ideas to new contexts.

Achievement of Unit Learning Outcomes

Learning outcome 1 Analyse computational problems

P2 compares selection, insertion and bubble sort through the costs of array reads and writes. I learned that algorithms with the same quadratic worst-case bound may have different practical operation costs. P4 analyses a larger engagement-score program by combining sorting and searching costs. P6 shows why splitting into more sublists does not automatically improve performance when merging is sequential. In Perfect Test, recognising that every retained prefix must remain non-negative led me to process values in their original order and use a min heap to support removal decisions.

Learning outcome 2 Implement use and test data structures

My C++ work implements and uses linked structures, sorting algorithms, heaps, trees and hash tables. P6 records tests for sorted input, reverse order, duplicates and negative values. C1 applies templates to reusable implementations and comparisons for custom types. The accepted 100/100 results for both 4K tasks add external test evidence. I understand that acceptance covers the judge tests, while readability, maintainability and broader correctness still require code review and clear reasoning.

Learning outcome 3 Design modular algorithmic solutions

C2 combines a generic linked list of batches with a binary search tree for each batch. Sorting prepares each batch for tree construction, and threshold searches produce candidates that are compared across batches. P8 combines a max heap for trending posts with a search tree for usernames. Faulty Check-in separates FIFO queue operations from LIFO stack operations. These designs show how I assign clear responsibilities to abstract data types and combine them to meet a larger requirement.

Achievement of Unit Learning Outcomes continued

Learning outcome 4 Use relevant tools and technologies

P1 records the clang++ compilation command and how to run the executable. My Git history records additions, refactoring and improvements to comments, including work on the 4K solutions. The 4K reports organise handwritten notes, judge verdicts and video links as evidence that can be inspected alongside the source. These practices help me reproduce work and communicate changes. When using AI assistance, I need to check suggestions against the implementation and remain responsible for understanding the algorithm.

Learning outcome 5 Plan monitor and reflect on development

P0 records my 4K target and why data structures and algorithms matter to my future studies. Weekly folders, task reports and repository revisions provide a record of development. P10 reflects on applied and Buddy Group activities. My Perfect Test notes, reconstructed from my recollection of the first attempt, show a specific revision: I corrected a misunderstanding of the requirement and changed the heap strategy. I can improve my monitoring by documenting the reason for each change and the tests used to check it.

Learning outcome 6 Communicate technical reasoning

P7 explains div_3 through initialisation, maintenance and termination of a loop invariant. C2 explains the branches in its threshold search. HD1 counts lifetime pushes and pops to justify linear total stack work, while HD2 distinguishes expected logarithmic tree height from a possible linear worst case. My 4K reports add worked traces and the new problems Single-Lane Cargo Station and A Day Full of Activities. These examples show how I communicate correctness, performance assumptions and implementation choices through prose, diagrams and code.

Reflections

Most important thing learnt

The most important thing I learnt is to translate the problem constraints into a property the algorithm must preserve. Perfect Test made this concrete. My initial approach focused on the final total and used a max heap, but the score must stay non-negative at every point and retained questions must keep their order. Processing scores sequentially and removing the lowest selected score when necessary addresses those constraints. This experience changed my approach: I now try to state the required invariant before deciding how to implement a solution.

Most interesting topic or task

Perfect Test was especially interesting because a local removal decision supports a larger optimisation goal. A min heap identifies the lowest selected score, and removing it gives the greatest increase in the current total among single-removal choices. The heap manages the selected scores without determining their chronological order. Developing A Day Full of Activities helped me express the same idea in terms of energy gained and consumed. I gained a clearer understanding of how an algorithmic structure can remain useful when the surrounding story changes.

Challenges Overcome

One challenge was distinguishing what a structure stores from the order in which the problem must be processed. My reconstructed Perfect Test notes show how rereading the statement and tracing a small example helped me replace the initial max-heap approach with a min heap. Faulty Check-in presented a related modelling challenge: the second line is described as a queue, but adding and removing at its front gives LIFO behaviour. Writing down the permitted operations made the stack model clear. These tasks taught me to work from operational rules and use traces to test an interpretation before coding.

What next?

I want to learn more about balanced and randomised search trees, automated invariant checks and memory-safe C++ design. HD2 gives me a starting point for understanding rotations and the difference between expected and worst-case performance. My linked structures also make ownership and cleanup important. I want to build small automated checks for tree ordering, parent pointers and heap properties, so that later revisions can be checked more reliably than by inspecting printed output alone.

Reflections continued

My initial learning plan

My engagement contract in P0 set a 4K target and connected the unit to future study and software development. The weekly notes and progression through P, C, D, HD and 4K work gave my learning a practical structure. Completing the two 4K tasks and explaining their strategies represents progress towards that target. However, the timeline field in my engagement contract was left blank, so I cannot assess progress against a detailed original schedule. Next time, I would set milestones for implementation, testing and reflection, and record progress against them each week.

Retrospective Advice for Myself

I would tell myself to read every constraint carefully, trace one small example and write the invariant before choosing a data structure. This would have helped me identify the prefix requirement in Perfect Test earlier. I would also analyse the actual implementation rather than rely on the familiar name of an algorithm. P6 shows why the merge strategy matters, and HD1 shows why nested loops can still have linear total work. I would keep expected outputs with my test inputs and record why each revision was made.